

Skinet RAG Plan

Goal: add a portfolio-worthy Retrieval-Augmented Generation feature to Skinet that helps shoppers ask natural-language questions about products, policies, and shopping decisions.

1. Recommended First Use Case

Build an AI Shopping Assistant first.

Primary jobs:

- Answer product questions in plain language.
- Recommend products by budget, type, brand, or use case.
- Explain why a product was suggested.
- Answer simple policy questions using curated store documentation.

This is the best first RAG feature because it is useful, easy to demo, and safer than starting with autonomous actions.

2. Data Sources for Retrieval

Source	Use in RAG	Notes
Products	Main catalog retrieval	Name, description, price, brand, type, image URL
ProductBrands , ProductTypes	Extra product metadata	Helps with filtering and explanation
Store policy documents	Support answers	Shipping, returns, payment, delivery FAQs
DeliveryMethods	Shipping guidance	Useful for checkout and delivery questions

Do not put baskets, orders, or user-specific records into the first public RAG index. Those are better handled later with authenticated tool calls.

3. Architecture

- **Client:** add a chat panel or assistant widget in Angular.
- **API:** add an AI endpoint in the .NET API, for example `POST /api/ai/chat`.
- **Retriever:** build embeddings for product and policy documents, store them in a vector store.
- **Generator:** send the user question plus retrieved context to the model.
- **Response:** return answer text, cited products, and optional suggested actions.

4. Retrieval Strategy

- Chunk product and policy content into short searchable documents.
- Store document text plus metadata such as product id, brand, type, price, and URL.
- Use hybrid filtering when possible: semantic retrieval plus structured filters like price, brand, and type.
- Return top matching chunks and pass only relevant context to the model.

5. Suggested Output Shape

Return a structured response from the AI endpoint:

- `answer` : natural-language reply
- `products` : matched products with ids and display data
- `sources` : policy or catalog sources used
- `followUpSuggestions` : optional next prompts

6. Rollout Plan

Phase 1

- Add product and policy ingestion.
- Generate embeddings for product catalog and store policies.
- Expose a read-only assistant endpoint.
- Show cited products and suggested next actions in the UI.

Phase 2

- Add authenticated tool-based features for signed-in users.
- Support questions like “Where is my order?” or “What is in my basket?” via secure API tools, not public RAG documents.

7. Portfolio Value

- Shows full-stack work beyond CRUD.
- Demonstrates practical AI architecture, not just a generic chatbot.
- Lets you discuss embeddings, retrieval quality, security boundaries, and hallucination control in interviews.

8. Success Criteria

- Users can ask shopping questions in natural language.
- The assistant returns relevant products and grounded explanations.
- Policy answers come from approved documents rather than free-form guessing.

- The first release is read-only and safe for demo and portfolio use.

9. Recommended Next Build Slice

Implement first: product catalog + store policy RAG assistant.

Skip agentic actions in v1. Add order and basket tool-calling only after the read-only assistant works well.